

s2.02 — Certificate / trust-bundle distribution, automatic renewal, and safe CA rollover

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Renewal and deployment fencing — IC3 baseline](#)
- [CA rollover and adoption evidence — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: JD fit plus OCI Certificates documentation, recorded in [the bank](#).
 - The architecture below is a proposed interview answer.
 - OCI product behavior is labeled `documented OCI behavior`
 - Everything else is proposed design.
- Why this is hard
 - Naive: a cron job renews certificates, copies PEM files to every host, and swaps the CA certificate on a Friday.
 - First break: a leaf signed by the new CA reaches a client whose trust store lacks that CA, and every handshake to that service fails at once.
 - Second break: the control plane cannot force consumers to switch
 - A retried renewal issues two versions
 - A stale rollout overwrites a newer one
 - Nobody knows which serial is serving traffic.
 - Minimum primitives
 - Immutable versions
 - An idempotent scheduler
 - Deployments fenced by a generation with per-target acks
 - Data-plane adoption evidence
 - A rollover state machine that orders trust before leaves.
- 🗣️ "Renewal is a signing job; rollover is coordination with consumers you do not control, so distribute trust first, switch leaves second, and retire old trust only on serving evidence."
- Opening script
 - "I will treat this as three control-plane workflows over one data plane: automatic renewal, versioned distribution, and CA rollover."

- “First I confirm who the consumers are, the leaf and CA lifetimes, and whether the service or the customer holds the private keys.”
- “Then I show why the numbers make this a correctness problem, and define an API where every version is immutable and every deployment is fenced by a generation.”
- “The deep dives are idempotent renewal at two-thirds lifetime and the five-phase rollover gated on adoption evidence.”
- “I will close with static stability during a control-plane outage, rollback limits, and what changes for a root rollover.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Consumers, lifetimes, key custody, primary invariant
Estimates and contract	11:00–17:00	Renewal rate, rollover burst, poll load; API semantics
Architecture	17:00–30:00	Diagram, control/data-plane split, one renewal trace
Deep dive	30:00–45:00	Idempotent renewal, generation fencing, rollover state machine
Operations and tradeoffs	45:00–50:00	Failure table, static stability, rollback limits, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Who consumes certificates?	Load balancers, gateways, sidecars, VM agents; pull and cache	Pull with ETag and a local cache is what makes the data plane statically stable
Leaf and CA lifetimes?	Leaf 90 d; subordinate CA 5 y; root 20 y (assumed)	Fixes renewal cadence, overlap windows, and how rare a rollover is
Who holds private keys?	CA keys in Vault HSM by handle; leaf keys managed or customer CSR	Managed keys can be rekeyed; customer keys need a CSR workflow and a deadline
Targets per certificate?	Up to 1,000 in a target group	Fan-out, ack table size, rollout batching
Partial rollout acceptable?	Yes, staged with canary; every step rollback-able	Rollout policy, generation fencing, rollback as a forward transition
Slow or offline verifiers?	Yes, some appliances update trust manually	Adoption evidence per cohort; retirement needs explicit exemptions

- Functional scope
 - Renew certificates automatically into a new immutable version.
 - Version trust bundles and distribute them ahead of any leaf switch.
 - Deploy explicit certificate and bundle versions to target groups with staged rollout and acks.
 - Collect adoption evidence: downloaded, loaded, serving.
 - Roll a subordinate or root CA over with an ordered, resumable state machine.
 - Roll back a deployment to a still-valid previous version.

Requirement	Target	Enforced by
Renewal lead time	Enqueue at 2/3 lifetime; 90 d leaf renews on day 60	Scheduler predicate over an expiring-versions index

Requirement	Target	Enforced by
Expiry SLO	0 auto-renew certificates reach notAfter while CURRENT	Page at 14 d to expiry; escalate at 7 d
Overlap	Both leaf versions valid ≥ 7 d; old CA trust kept ≥ 30 d after leaves switch	Version state machine and rollover gates
Assignment read	p99 < 50 ms, 99.99%	ETag on signed manifests; artifacts by hash from object storage or CDN
Convergence	95% of targets LOADED within 15 min; 100% or exemption before retirement	Ack table joined against the target registry
Static stability	Drill 24 h only with > 24 h remaining required validity	Cached leaf, chain, key, and acceptable status; nominal 30 d renewal margin is not a fleet-wide guarantee
Control-plane availability	99.9% for lifecycle APIs	Stateless API fleet; replicated metadata store
Rollback	Previous version serving within 5 min while unexpired and unrevoked	Rollback is a new higher generation, never a decrement

- Invariants
 - Trust before leaves: no target serves a leaf whose issuer is missing from the bundle deployed to its verifiers.
 - Versions are immutable
 - A deployment references explicit versions, never latest.
 - One authoritative deployment generation per target group
 - Acks with an older generation are rejected.
 - Renewal of (certificate, base version) produces at most one new version.
 - Old trust is retired only after serving evidence shows zero old-chain handshakes in every cohort.
 - CA keys stay in the HSM by handle
 - A manifest carries public material and a key-delivery reference only.
- Non-goals
 - Revocation and CRL/OCSP distribution (s2.03).
 - Issuance policy and CSR validation (s2.01).
 - Root key ceremony and HSM operations (s2.04).

Capacity and consequence — IC3 baseline

- Renewals
 - 1M leaves renewed every 60 d is about 16.7k per day, roughly 0.2/s.
 - A 5 min scheduler tick selects on the order of 60 rows.
 - Decision: renewal is a correctness workload
 - One leased scheduler with idempotent enqueue is enough, and the clock goes to fencing and rollover.
- Rollover burst
 - Rolling a subordinate CA with 100k leaves re-issues all of them.
 - HSM asymmetric signing is order of magnitude hundreds per second
 - 100k / 100 per s ≈ 17 min of signing, closer to 1 h with policy checks and commits.
 - Decision: SWITCH_LEAVES is a queued, rate-limited batch with jitter that lasts hours to days by design.
- Distribution reads
 - 200k targets polling hourly with ETag is about 55 req/s, mostly 304.
 - A fleet restart reloading 200k targets in 5 min is about 670 req/s
 - At 8 KB per manifest that is about 5 MB/s.
 - Decision: pull with jitter and ETag
 - Push events only hint to poll sooner
 - Bundle bytes live in object storage or a CDN by content hash.

- Acks and bundles
 - 200k targets at about 200 B per ack row is 40 MB per fleet-wide deployment
 - Three retained generations is order of 100 MB.
 - Five CAs at about 2 KB PEM each is 10 KB
 - Overlap plus a cross-certificate doubles it to 20 KB.
 - Decision: acks are overwritten in place per (deployment, target)
 - Bundle contents are immutable by sha256 and verified before load.
- Adoption evidence
 - 200k targets emitting per-serial counters every 60 s is about 3.3k datapoints per second.
 - Decision: aggregate at the agent as {issuer SKI, leaf serial} → count per minute
 - Ship counters, not handshake events.

Architecture and flow — IC3 baseline

```
[control plane]
Operator / CI -> API gateway (IAM policy, compartment) -> lifecycle API -> metadata DB
                                                    (certs, versions, bundles,
                                                    deployments, acks, audit)

Renewal scheduler --(2/3 lifetime, idempotent enqueue)--> queue --> issuance worker --> Vault / HSM Sign
Rollover orchestrator --(per-CA state machine)--> deployments (generation g, g+1, ...)
Bundle publisher --> object storage (immutable, keyed by sha256) --> CDN

[data plane]
Consumer agent (LB / gateway / sidecar)
  GET assignment (If-None-Match) -> signed manifest
  -> fetch artifacts by hash -> verify chain against bundle -> hot-swap TLS context
  -> PUT ack {generation, DOWNLOADED | LOADED | SERVING | FAILED}
  -> handshake counters by issuer SKI and serial -> adoption evidence store
  TLS handshakes never call the control plane; last-known-good lives on local disk
```

- Lifecycle API and metadata store own desired state.
 - Mechanism bought: one source of truth for immutable versions
 - Mutations return 202 with a work-request identity.
 - Cost paid: the certificate or target-group row is the consistency boundary, so every version flip and generation bump serializes on it.
- Renewal scheduler owns "when".
 - Mechanism bought: a partitioned scan of the expiring-versions index with a dedup identity per (certificate, base version).
 - Cost paid: one leased leader
 - A stalled scheduler is invisible until the 14 d alarm, so scheduler lag is itself a metric.
- Issuance worker and Vault HSM own "sign".
 - Mechanism bought: the worker sends a digest and receives a signature
 - Documented OCI behavior is that HSM-protected keys cannot be exported.
 - Cost paid: HSM throughput caps rollover speed and forces queueing with jitter.
- Rollover orchestrator owns ordering.
 - Mechanism bought: a persisted per-CA state machine whose transitions are gated on evidence.
 - Cost paid: slow by design
 - The last gate needs a human to own exemptions.
- Bundle publisher, object storage, and CDN own bytes.
 - Mechanism bought: content-addressed immutable artifacts that consumers verify offline.
 - Cost paid: CDN TTL bounds propagation, so freshness lives in the manifest, not the artifact.
- Consumer agent owns "load".
 - Mechanism bought: pull, verify, hot-swap the TLS context, ack, emit counters

- The process never restarts.
- Cost paid: agent version skew is a compatibility surface
 - The manifest schema is a frozen contract.
- Adoption evidence store owns "proof".
 - Mechanism bought: counters per (target, issuer SKI, serial, minute) answer "who still serves the old chain".
 - Cost paid: a silent cohort blocks retirement, so targets that stop reporting must be enumerated.

→ Scheduler selects a `CURRENT` version past 2/3 lifetime with no successor and enqueues `renew:{cert}:{version}`.

→ Worker locks the certificate row, re-checks that no successor exists, builds the CSR or waits for a customer CSR.

→ Worker calls Vault `sign` on the CA key handle and inserts version `n+1` as `PENDING`.

→ Worker flips `n+1` to `CURRENT` and `n` to `PREVIOUS` in one transaction and publishes `CertificateRenewed`.

→ Orchestrator creates a deployment at generation `g+1` referencing version `n+1` and the current bundle version.

→ Agents poll, see the new ETag, fetch artifacts by hash, and pre-flight the chain against the deployed bundle.

→ Agents hot-swap the TLS context and ack `LOADED` with generation `g+1`; new handshakes report serial `n+1`.

→ Deployment reaches `CONVERGED` on acks; version `n` becomes `DEPRECATED` only after counters show serial `n` at zero.

API and data model — IC3 baseline

Surface	Contract	Idempotency	Conflict/error behavior
POST <code>/v1/certificates/{id}/renewals</code>	<code>{baseVersion, csrPem?}</code> returns 202 <code>{newVersion, operationId}</code>	I on <code>(certificate, baseVersion)</code>	409 <code>VERSION_SUPERSEDED</code> if a successor exists
POST <code>/v1/trust-bundles/{id}/versions</code>	<code>{issuerRefs, reason}</code> returns 201 <code>{bundleVersion, sha256}</code>	I	Contents immutable; identical contents return the existing version
POST <code>/v1/certificate-deployments</code>	<code>{certificateVersion, trustBundleVersion, targetGroup, rolloutPolicy}</code> returns 202 <code>{deploymentId, generation}</code>	I	409 <code>STALE_REFERENCE</code> if a version is deprecated, revoked, or expired
GET <code>/v1/certificate-deployments/{id}</code>	<code>{state, desired, applied, failed, pending, silent}</code>	read	Convergence is computed from acks, not from the create call
Agent GET <code>/v1/certificate-assignments/{target}</code>	If-None-Match returns 200 signed manifest or 304	read	Agent identity maps to one target; public artifacts plus a key-delivery reference only
Agent PUT <code>/v1/certificate-deployments/{id}/acks/{target}</code>	Generation, assignment hash, sequence, load state, health	Replay of an old sequence is a no-op	Reject any generation/hash mismatch; health may later degrade
POST <code>/v1/certificate-deployments/{id}/rollbacks</code>	<code>{targetVersion, reason}</code> returns 202 <code>{generation}</code>	I	409 <code>UNRECOVERABLE</code> if the target version is revoked, expired, or its issuer retired
POST <code>/v1/ca-rollovers/{id}:advance</code>	<code>{expectedPhase}</code> returns 202	compare-and-set on phase	409 <code>GATE_NOT_MET</code> with the failing gate and the silent-target list

State	Key	Fields/invariant
Certificate version	(tenant_id, cert_id, version_num)	PENDING, CURRENT, PREVIOUS, DEPRECATED, REVOKED, FAILED; serial, chain, validity; one CURRENT
Trust bundle version	(tenant_id, bundle_id, bundle_version)	manifest_ref, sha256, created_at; contents immutable
Deployment	(tenant_id, deployment_id)	Version refs, target_group, generation, state, policy; generation strictly increases per group
Target ack	(tenant_id, deployment_id, target_id)	generation, state, loaded_version_ref, observed_at, error_code
Rollover	(tenant_id, ca_id, rollover_id)	phase, old and new CA versions, bundle before/with-both/after, exemptions with owner and expiry
Adoption evidence	(target_id, issuer_ski, leaf_serial, minute)	Handshake count; retained 30 d; rolled up per cohort

- Reference DDL
 - Keys and uniqueness match the state table above and the [companion schema](#).
 - Adoption evidence counters live in the metrics store keyed by (target_id, issuer_ski, leaf_serial, minute); only versions, bundles, groups, deployments, acks, and rollovers are relational.

```

CREATE TABLE certificate_versions (
  tenant_id text NOT NULL, cert_id text NOT NULL, version_num bigint NOT NULL CHECK (version_num >= 1),
  stage text NOT NULL
  CHECK (stage IN ('PENDING','CURRENT','PREVIOUS','DEPRECATED','REVOKED','FAILED')),
  serial text, issuer_ca_version_ref text NOT NULL,
  chain_ref text, -- chain artifact by hash in object storage
  not_before timestampz, not_after timestampz,
  key_ref text, -- Vault/HSM handle, never key material
  created_at timestampz NOT NULL, version bigint NOT NULL,
  PRIMARY KEY (tenant_id, cert_id, version_num), -- successor is version_num + 1, so one renewal per base
  version
  CHECK (not_before IS NULL OR not_after > not_before)
);
CREATE UNIQUE INDEX certificate_versions_one_current ON certificate_versions (tenant_id, cert_id)
  WHERE stage = 'CURRENT';
CREATE UNIQUE INDEX certificate_versions_serial ON certificate_versions (issuer_ca_version_ref, serial)
  WHERE serial IS NOT NULL;
CREATE INDEX certificate_versions_expiring ON certificate_versions (not_after) WHERE stage = 'CURRENT';
CREATE TABLE trust_bundle_versions (
  tenant_id text NOT NULL, bundle_id text NOT NULL, bundle_version bigint NOT NULL CHECK (bundle_version >=
1),
  manifest_ref text NOT NULL, sha256 text NOT NULL, created_at timestampz NOT NULL,
  PRIMARY KEY (tenant_id, bundle_id, bundle_version),
  UNIQUE (tenant_id, bundle_id, sha256) -- identical contents return the existing version
);
CREATE TABLE target_groups (
  tenant_id text NOT NULL, target_group text NOT NULL,
  current_generation bigint NOT NULL CHECK (current_generation >= 0), -- bumped under this row's lock;
  never decrements
  owner_epoch bigint NOT NULL, -- orchestrator lease epoch; a resumed old owner fails the CAS
  PRIMARY KEY (tenant_id, target_group)
);
CREATE TABLE certificate_deployments (
  tenant_id text NOT NULL, deployment_id text NOT NULL,
  cert_id text NOT NULL, version_num bigint NOT NULL,
  bundle_id text NOT NULL, bundle_version bigint NOT NULL,
  target_group text NOT NULL, generation bigint NOT NULL,
  assignment_hash text NOT NULL, -- hash of the signed manifest agents must ack
  state text NOT NULL CHECK (state IN ('ROLLING','PAUSED','CONVERGED','SUPERSEDED','FAILED')),
  policy jsonb NOT NULL, created_at timestampz NOT NULL,
  PRIMARY KEY (tenant_id, deployment_id),
  UNIQUE (tenant_id, target_group, generation),
  FOREIGN KEY (tenant_id, cert_id, version_num) REFERENCES certificate_versions,
  FOREIGN KEY (tenant_id, bundle_id, bundle_version) REFERENCES trust_bundle_versions,
  FOREIGN KEY (tenant_id, target_group) REFERENCES target_groups
);
CREATE TABLE certificate_target_acks (
  tenant_id text NOT NULL, deployment_id text NOT NULL, target_id text NOT NULL,
  generation bigint NOT NULL,
  sequence bigint NOT NULL DEFAULT 0 CHECK (sequence >= 0), -- replay of an older sequence is a no-op
  state text NOT NULL CHECK (state IN ('ASSIGNED','DOWNLOADED','LOADED','SERVING','FAILED')),
  loaded_version_ref text, health text, error_code text,
  observed_at timestampz NOT NULL, -- no ack for 24 h makes the target silent, not done
  PRIMARY KEY (tenant_id, deployment_id, target_id),
  FOREIGN KEY (tenant_id, deployment_id) REFERENCES certificate_deployments
);
CREATE INDEX certificate_target_acks_rollout ON certificate_target_acks
  (tenant_id, deployment_id, state, target_id);
CREATE INDEX certificate_target_acks_silent ON certificate_target_acks (observed_at)
  WHERE state NOT IN ('SERVING','FAILED');
CREATE TABLE ca_rollovers (
  tenant_id text NOT NULL, ca_id text NOT NULL, rollover_id text NOT NULL,
  phase text NOT NULL CHECK (phase IN
  ('PUBLISH_TRUST','VERIFY_TRUST','SWITCH_LEAVES','VERIFY_SERVING','RETIRE_OLD_TRUST','DONE')),
  old_ca_version_ref text NOT NULL, new_ca_version_ref text NOT NULL,
  bundle_id text NOT NULL,
  bundle_before bigint NOT NULL, bundle_with_both bigint, bundle_after bigint,
  exemptions jsonb NOT NULL DEFAULT '{}', -- [{target_id, owner, expires_at}]

```

```
gate_results jsonb NOT NULL DEFAULT '{}', -- persisted per phase so advance is auditable
owner_epoch bigint NOT NULL, version bigint NOT NULL, updated_at timestamptz NOT NULL,
PRIMARY KEY (tenant_id, ca_id, rollover_id),
FOREIGN KEY (tenant_id, bundle_id, bundle_before) REFERENCES trust_bundle_versions
);
CREATE UNIQUE INDEX ca_rollovers_one_active ON ca_rollovers (tenant_id, ca_id) WHERE phase <> 'DONE';
```

- Not captured by the DDL
 - Adoption evidence is a per-minute counter series with 30 d retention and per-cohort rollups; it stays in the metrics store and is read by the `VERIFY_SERVING` gate, not joined in SQL.
 - Private keys, the manifest signing key, and certificate or chain bytes live in Vault, the HSM, and object storage behind `key_ref`, `chain_ref`, and `manifest_ref`.
 - Deviation from the companion: `certificate_versions`, `target_groups`, and `ca_rollovers` are added because the state table and the fencing deep dive name them; `certificate_deployments` references the version row by key instead of a free `certificate_version_ref`, gains `assignment_hash`, and `certificate_target_acks` gains `sequence` and `health`.
- Proposed service conventions
 - Manifest: {`generation`, `certificateVersionRef`, `chainHash`, `trustBundleRef`, `notBefore`, `notAfter`, `signature`}, signed by a service key in Vault that is not a CA key
 - Agents pin its public key at install.
 - The reference DDL above adapts the [API companion](#) tables for bundles, deployments, and acks; the deviation bullet lists every difference.
 - Documented OCI behavior: a CA bundle holds root and intermediate certificates in PEM, may include CAs not managed by the service, and a chosen version is marked current.
- Version stages are documented in the [OCI SDK model](#).
 - The deployment states and gates in this answer remain proposed behavior.

Renewal and deployment fencing — IC3 baseline

- Why 2/3 lifetime
 - A 90 d leaf renews on day 60, leaving 30 d to detect and fix a consumer that did not switch.
 - Thirty days is the nominal margin at the planned renewal instant.
 - The actual outage budget is the minimum remaining validity across active leaves, chains, and required status evidence.
 - Retries back off until day 75
 - Then the on-call is paged with 15 d of margin.
 - ±5% jitter stops a fleet issued on one day from renewing in lockstep against the HSM.
- Three layers of idempotency
 - The scheduler predicate selects only `CURRENT` versions with no `version_num + 1` row.
 - The queue deduplicates on `renew:{cert_id}:{version_num}`.
 - The worker locks the certificate row and re-checks the successor before signing.
 - Durable unique renewal intent and fenced publication provide correctness.
 - Scheduler filtering and queue deduplication only reduce redundant work.
- Managed versus customer-held key
 - Managed: the worker generates a new key pair in the tenant's Vault key or at the agent
 - A certificate renewal is also a key rotation.
 - Customer-held: the service cannot recreate the key from a certificate
 - Renewal waits in `AWAITING_CSR`, notifies the owner, and pages at day 75.
 - Persist `PENDING` with serial, issuer key version, and exact TBS before signing, as in s2.01.
 - Resume that intent after a crash rather than allocate an untracked replacement.
- Generation fencing

- Each target group has a monotonically increasing `generation`, bumped under the group row lock when a deployment is created.
- The agent ignores a manifest with a lower generation than it applied, which covers a stale CDN or lagging replica.
- The server accepts an acknowledgment only for the exact current generation and assigned content hash.
- Future generations and unassigned targets are rejected too.
- Rollback is a new, higher generation pointing at an older version
- A generation never decrements.
- Exact failure prevented: a resumed old orchestrator must pass both expected generation and current owner epoch before allocating a new deployment.
- A row lock alone serializes writers but does not identify a stale owner.
- Rollout policy
 - Canary one target, then `10%`, `50%`, `100%` of the group.
 - Each step waits for `LOADED` acks and an unchanged handshake error rate for `10 min`.
 - Abort creates a rollback generation automatically
 - The `PREVIOUS` version is still valid, so the fallback exists by construction.
- Agent pre-flight
 - Before loading, the agent builds the path from the new leaf to an anchor in the bundle version named by the manifest.
 - A failure acks `FAILED` with `CHAIN_UNTRUSTED` and keeps serving the old version.
 - This is the local enforcement of trust-before-leaves
 - It turns a fleet outage into one failed ack.
- Acknowledgment flow
 - Validate the target assignment before counting application state.
 - Track health independently so a loaded target can later report failure.

```

ACK(deployment, target, generation, assignment_hash, sequence, load_state, health):
    transaction:
        lock deployment and target observation
        require authenticated target belongs to deployment
        require generation == current assigned generation
        require assignment_hash == target's current assignment hash
        if sequence <= last_observation_sequence: return stored observation
        persist sequence, load_state, health, observed_at
        derive rollout counts from current-generation target rows
        pause on unhealthy observation according to rollout policy
        converge only when all required assignments and health gates pass
        append audit/outbox with the observation

```

CA rollover and adoption evidence — IC3 baseline

PUBLISH_TRUST gate -> approved	bundle b+1 = old CA + new CA (+ cross-cert); deploy to every group that verifies this CA every required verifier cohort loaded the exact assignment; exclusions explicitly
VERIFY_TRUST gate ->	one canary leaf under the new CA on a canary group; old chain keeps serving elsewhere canary handshake error-rate delta < 0.1% for 1 h across client cohorts
SWITCH_LEAVES deployment)	re-issue every CURRENT leaf under the new CA in jittered batches (normal renewal +
gate ->	every leaf under the old CA re-issued, or exempt with an owner and an expiry
VERIFY_SERVING gate ->	handshakes by issuer SKI; old-CA serials must reach zero zero old-chain handshakes for 72 h in every cohort, not in the aggregate
RETIRE_OLD_TRUST signing until its end ->	bundle b+2 = new CA only; deploy; old CA key remains authorized only for required CRL last leaf expires old CA version DEPRECATED; key disabled, then scheduled for deletion after the waiting period

- Transition rules
 - Every phase and gate result is persisted on the rollover row
 - `advance` is a compare-and-set on `expectedPhase`.
 - Phases move forward only
 - "back" means a new bundle or deployment generation that restores the previous phase's contents.
 - Restoring an old trust anchor requires an unexpired, uncompromised certificate accepted by current policy.
 - Deleting its private key prevents further signing, not mathematical verification of existing signatures.
- Cross-signing
 - The old CA signs a certificate for the new CA's public key, so one key carries two issuer signatures.
 - Test alternative paths on representative verifiers.
 - A cross-certificate does not guarantee every path builder can reach either anchor.
 - AKI/SKI must be correct on both intermediates or path building picks the wrong parent and reports `unknown issuer`.
 - The cross-certificate leaves the served chain only after `RETIRE_OLD_TRUST`.
 - Compatibility tests cover path length, name constraints, algorithm support, intermediate delivery, and anchor expiry.
- Evidence ladder
 - Downloaded: the agent fetched and hash-verified the artifact
 - Proves distribution.
 - Loaded: the agent acked after a hot swap
 - Proves the target can serve.
 - Serving: counters show the new serial at servers and the new issuer SKI at clients
 - Only this proves traffic moved.
 - A deployment `CONVERGED` on acks but showing old serials means an agent loaded a context the listener is not using.
- Silent cohort
 - A registered target with no ack or counters for `24 h` is `silent`, not done.
 - A gate cannot pass with unknowns
 - An exemption names an owner, a reason, and an expiry, and is audited.
 - Concrete values: CA overlap ≥ 30 d, serving window `72 h`, canary soak `1 h`.
- Static stability of consumers
 - The agent persists manifest, artifacts, and bundle on disk and loads them at boot before any network call.
 - A `5xx` or timeout on poll leaves the current context untouched
 - The agent backs off with jitter.
 - The cache extends availability, never validity
 - The outage countdown is the earliest `notAfter` in the fleet.
 - Defeater to volunteer: a consumer that fetches on boot with no local copy dies in a control-plane outage, so last-known-good ships in the image or persists on disk.
- Rollback limits
 - A leaf rollback works while the previous version is unexpired and unrevoked.
 - A bundle rollback requires a still-valid and uncompromised old trust anchor.
 - Private-key availability is needed for future signatures and CRLs, not for verifying existing signatures.
 - After `RETIRE_OLD_TRUST`, re-adding the old CA is a new bundle version, which is a forward rollover in the other direction.
 - Last-known-good caching never revives a revoked or expired trust state.
- Exact failure
 - Switching leaves before trust is loaded yields `unknown issuer` at every client lacking the new CA.
 - The failure appears one hop away from the server that received the certificate, which is why client-side counters and the agent pre-flight both exist.
 - Do not depend on automatic AIA fetching across client runtimes.

- Test the configured JVM and OpenSSL clients with exactly the chain delivered by the server.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Scheduler leader dead	Renewal backlog	Intent age and nearest validity deadline	Existing material remains usable only until its actual expiry/status deadline	Fenced failover and durable-intent catch-up
Duplicate renewal enqueue	One certificate	Dedup hit counter	Row lock and successor check make the second a no-op	None; alert if dedup misses rise
Vault or HSM unavailable	New issuance and rollover	<code>Sign</code> error rate	Serving continues; renewals back off; expiry countdown	Vault fails over across availability domains; page at 14 d
Consumer never adopts	That target's clients	Ack missing past SLA; counters show old serial	Deployment stalls; old version still valid	Inspect agent; fix or exempt with expiry
Leaf switched before trust (operator override)	One service's clients	<code>CHAIN_UNTRUSTED</code> acks; client handshake errors	Agent refuses to load; keeps old version	Rollback generation; re-run <code>PUBLISH_TRUST</code>
Two orchestrators after lease expiry	One target group	<code>409 STALE_GENERATION</code> rate	Stale acks rejected; stale bump loses the lock	Renew lease; reconcile deployments
Corrupt or stale bundle artifact	Polling targets	<code>sha256</code> mismatch at agent	Agent keeps last-known-good	Republish; purge CDN; new bundle version
Control plane or metadata store outage	Lifecycle operations	Availability and remaining-validity alarms	Cached consumers continue only within their actual certificate/status validity	Restore authority and resume incomplete intents
Old CA retired while an offline appliance serves the old chain	That appliance's clients	Per-cohort serving evidence	Clients without old trust reject it	Re-add old CA in a new bundle if unexpired; else re-issue the appliance leaf

- Observe
 - Certificates expiring within 14 d and 7 d, by tenant and CA.
 - Renewal success rate, scheduler lag, dedup hits.
 - Convergence time, failed acks by error code, silent target count.
 - Handshake counters by issuer SKI and serial
 - Old-chain share during a rollover.
 - Rollover phase age
 - A phase older than its planned window pages.
- Retry discipline
 - Exponential backoff with jitter on `Sign`, queue, and poll paths.
 - Renewal retries are bounded by the day 75 page, not by attempt count.
 - `409 STALE_GENERATION` and `CHAIN_UNTRUSTED` are corrective, never retried blindly.
 - A generation bump that loses the row lock re-reads and rebases rather than retrying the same value.
- Tenant isolation
 - Every certificate, bundle, deployment, and target lives in a compartment
 - IAM policy statements gate create, deploy, and rollback.
 - Agent identity is a resource principal mapped to one target
 - It reads its own assignment and writes its own ack only.
 - Quotas cap deployments in flight and targets per group per tenant.
 - A tenant's token receives `404` on another tenant's identifiers, not `403`, to avoid existence leaks.
- Rollout and rollback safety
 - Manifest schema changes are additive

- Old agents ignore unknown fields.
- A new orchestrator must accept acks from the previous agent version.
- Bundle changes are canaried like leaf changes because a bad bundle has the widest blast radius in the system.
- Chaos test: isolate agents for 24 h after checking the entire test cohort has > 24 h remaining required validity.
- Include separate near-expiry cases that must follow the declared fail policy.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Push distribution	Control plane writes material to each consumer	Fast propagation	Must reach every host; a push outage halts rotation; no static stability by default	Small fleet inside one network
Pull with ETag	Agent polls a signed manifest and fetches by hash	Statically stable; CDN-scalable; agent authenticates itself	Propagation bounded by poll interval	Baseline; hint with events
Hard cutover	Swap CA and leaves in one step	Simple to explain	Every cached client breaks at once	Never in production
Overlap with cross-signing	Alternate certified path for the new key	Can bridge compatible old-anchor clients	Path-builder and constraint compatibility; not safe after old-key compromise	Planned rollover after testing every required client cohort
Short-lived leaves	24 h certificates; expiry replaces leaf rollover	Leaves roll over by themselves within a day	Issuance becomes a hot path with its own SLO	Service mesh with automated identity

- IC4 stretch — stronger requirement: root rollover across verifiers you do not control
 - Customer devices, Oracle wallets, and JVM cacerts run no agent, so serving evidence is incomplete by construction.
 - Build a verifier inventory from LB and gateway registries and client cohorts before the rollover, not during.
 - Retire old trust only when every mandatory verifier cohort is covered or an explicit owner accepts the remaining outage risk.
 - A fleet-wide percentage alone cannot prove compatibility.
- IC4 stretch — 10× load
 - 10M certificates at 24 h lifetime renew about 6.9k per minute
 - HSM sign becomes the bottleneck.
 - Delegate per-node issuers with short-lived, name-constrained sub-CAs so the HSM signs one sub-CA per node per day, not one leaf per pod.
 - Serve assignments from regional replicas and CDN-cached manifests
 - Batch acks per node.
- IC4 stretch — cost and blast radius
 - Cell the orchestrator and evidence store per region and per CA
 - One CA's rollover cannot stall another's renewals.
 - Two-tier hierarchy keeps root rollover rare
 - Most rollovers touch one subordinate and its 100k leaves.
 - Evidence retention dominates storage cost
 - Keep raw counters 30 d and rollups 7 y.
- IC4 stretch — migration
 - Adopt the agent on a legacy fleet in observe-only mode that reports counters without loading, so the evidence ladder exists before the first automated switch.
- Weakest assumption
 - Telemetry completeness: that every verifier is registered and reports.

- Mitigation: reconcile the target registry against LB and gateway inventories daily
 - Gate per cohort
 - Every exemption expires.
- Over-engineering warning
 - Do not build push distribution for 200k targets
 - Pull with a CDN is simpler and statically stable.
 - Do not try to make trust publication and leaf switch atomic
 - The ordering is the safety property.
 - Do not add OCSP to solve rollover
 - Revocation is s2.03.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Consumers, lifetimes, key custody, trust-before-leaves invariant	Names the unregistered-verifier cohort and what excluding it costs	Asks who verifies, not only who serves
Estimates	Renewal rate, rollover burst against HSM throughput, poll load	Uses the burst number to size SWITCH_LEAVES duration and jitter	A number sets a phase length
Architecture	Control/data-plane split, pull with cache, one renewal trace	Cost of each component; evidence store as a first-class owner	Every component has a mechanism and a price
Deep dive	Idempotent renewal, generation fencing, five-phase rollover	Cross-sign chain mechanics, per-cohort gates, rollback limits	Explains why downloaded is not serving
Operations	Failure table, static stability, expiry alarms	Silent cohorts, expiring exemptions, data-plane chaos test	Volunteers what blocks retirement before being asked
Wrap	Recap and tradeoff	Names telemetry completeness as the weakest assumption	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: The renewal job ran twice.
 - **IC3 baseline**
 - The successor check, the queue dedup identity, and the worker's row lock each make the second run a no-op.
 - **IC4 stretch**
 - Name the crash between sign and insert
 - A retry mints a new serial, so the orphan is bounded by lifetime and reconciled as revoked.
- Q2: Issuance succeeded.
 - Is the rotation done?
 - **IC3 baseline**
 - No
 - The deployment converges on LOADED acks and version n is deprecated only when counters show serial n at zero.
 - **IC4 stretch**
 - A CONVERGED deployment with old serials in counters means the listener is not using the loaded context.
- Q3: A late ack from the previous rollout arrives after the new one started.
 - **IC3 baseline**
 - It carries the old generation and is rejected with 409 STALE_GENERATION.
 - **IC4 stretch**
 - The agent also ignores manifests below its applied generation, covering a stale CDN in the other direction.

- Q4: Why not switch leaves the same day trust is published?
 - **IC3 baseline**
 - Verifiers load trust on their own poll schedule
 - The agent pre-flight refuses a leaf whose issuer is not in the loaded bundle.
 - **IC4 stretch**
 - The `VERIFY_TRUST` canary catches verifiers without an agent, whose failure appears only in client error rates.
- Q5: The control plane is down for a week.
 - **IC3 baseline**
 - Cached material avoids a live issuance dependency.
 - The actual outage budget is the shortest remaining leaf, chain, or status-validity window.
 - **IC4 stretch**
 - The countdown is the earliest `notAfter` in the fleet
 - A consumer with no on-disk copy that restarts is the case that breaks static stability.
- Q6: Can we roll back after retiring the old CA?
 - **IC3 baseline**
 - Only while the old trust anchor remains valid, uncompromised, and permitted by current policy.
 - Private-key deletion affects future signing, not verification of an existing signature.
 - Rollback is a new bundle version that re-adds it.
 - **IC4 stretch**
 - That is why the key stays enabled through the CRL-signing tail and deletion after the waiting period is the last irreversible step.
- Q7 (premise reversal): Why need consumer acks at all?
 - The service is the source of truth
 - Mark the deployment done when the API call succeeds.
 - **IC3 baseline**
 - The API call proves desired state, not observed state
 - The canonical rotation outage is a certificate issued and never loaded.
 - **IC4 stretch**
 - Acks are only the middle rung
 - Retirement gates on serving evidence, and for verifiers without an agent the honest answer is an inventory plus a long cross-signed overlap.
- Q8: The customer holds the private key.
 - **IC3 baseline**
 - The service cannot recreate it, so renewal waits in `AWAITING_CSR` with a notification and a day 75 page.
 - **IC4 stretch**
 - Offer a managed-key migration where the next key is generated inside the tenant's Vault so later renewals are automatic.
- Validation scenarios
 - Double enqueue and worker crash-and-retry for the same base version.
 - Late ack with a stale generation.
 - Manifest with a lower generation served from a stale CDN.
 - Leaf deployed before its issuer is in the bundle
 - Agent pre-flight path.
 - Rollover gate with one silent target and one expired exemption.
 - Old CA retired while a cohort still reports old-issuer handshakes.
 - Control plane unreachable for 24 h with an agent restart in the middle.
 - Rollback requested after the previous version's `notAfter`.

Recall card

- “Immutable versions, explicit references, and a generation per target group; nothing points at latest.”
- “Renew at two-thirds lifetime so the control plane can be down a third of a lifetime before anyone notices.”
- “Downloaded is not loaded, and loaded is not serving; only handshake counters retire old trust.”
- “Publish trust, verify trust, switch leaves, verify serving, retire old trust; forward only, cross-signed so verifiers switch on their own schedule.”
- “The data plane serves last-known-good from disk; the cache extends availability, never validity.”
- Wrap-up script
 - “The design is a control plane of immutable certificate and bundle versions, an idempotent renewal scheduler, and fenced deployments that consumers pull, verify, and acknowledge.”
 - “The key invariant is trust before leaves, enforced by a five-phase rollover whose gates are per-cohort serving evidence.”
 - “The main tradeoff is pull with acknowledgments over push: slower propagation, but a data plane that survives a control-plane outage and a retirement gate that proves traffic moved.”
 - “My weakest assumption is telemetry completeness; I would reconcile the target registry against the load-balancer inventory before the first rollover.”

References

- [Ranked question and provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- Reference check: 2026-09-09.
 - Official documentation supports the mechanisms cited next to each link.
 - Capacity values, phase names, gate thresholds, and the deployment/ack model remain proposed interview assumptions.
- [OCI Certificates overview](#)
 - Bundle contents in PEM
 - CAs not managed by the service
 - Automatic renewal rules
 - Marking a version current
 - Load Balancer association
 - Import of external CAs and certificates.
- [OCI Vault key overview](#)
 - HSM-protected keys cannot be exported
 - Rotation creates a new key version while older versions stay usable
 - RSA and ECDSA keys sign
 - Key copies span availability or fault domains within a region.
- [OCI IAM policies](#): compartment-scoped policy statements behind the tenant isolation bullets.
- [RFC 5280](#): path validation and AKI/SKI linkage that make cross-certificates validate against either root.
- [Let's Encrypt DST Root CA X3 expiration](#): the canonical cross-sign and old-verifier failure the rollover gates are designed around.
- [Multi-tenant CA sibling design](#): version state machine, HSM custody, and the 2/3 renewal predicate reused here.
- [PKI and KMS mechanisms](#): X.509 extensions, chain building, KMS `Sign` with a digest, and the intermediate and root rotation sequences.